

Warmup: mapping reductions

$$3\text{-COLOR} \leq_p 4\text{-COLOR}$$

$$G = (V, E) \mapsto G' = (V \cup \{s\}, E \cup \{(s, v) : v \in V\})$$

$$\text{PALINDROME} \leq_p 3\text{-COLOR}$$

$$x \mapsto 3\text{-clique if palindrome, } 4\text{-clique otherwise.}$$

$$3\text{-COLOR} \leq_p \text{HALT}$$

★ **Fact:** 3-COLOR is NP-complete

Corollary: if $3\text{-COLOR} \leq_p \text{PALINDROME} \Rightarrow \text{P} = \text{NP}$

Pf: to show $\text{P} = \text{NP}$, exhibit a poly-time algorithm for arbitrary $L \in \text{NP}$ under the assumed reduction

Fixing $L \in \text{NP} \Rightarrow \exists$ poly-time $R_{L \rightarrow 3\text{COL}}$ by ★

Given instance x to L :

- run $y \leftarrow R_{L \rightarrow 3\text{COL}}(x)$
- run $z \leftarrow R_{3\text{COL} \rightarrow \text{PAL}}(y)$ by assumption
- solve $z \in \text{PALINDROME}$
 - if yes, accept
 - if no, reject $\square$

Cook–Levin Theorem

SAT is NP-complete.

↳ **Karp** showed an incredibly diverse set of problems Π with $\text{SAT} \leq_p \Pi$.

SAT: Boolean satisfiability

Boolean formula in conjunctive normal form

↳ n boolean variables $x_1, \dots, x_n$

$$m \text{ clauses } c_j = (\ell_{j_1} \vee \ell_{j_2} \vee \dots \vee \ell_{j_k})$$

↳ $m = \text{poly}(n)$

$$\ell_i \in \{x_i, \neg x_i\}$$

$$\phi = \bigwedge_{j=1}^m c_j$$

SAT: does there exist an assignment $a \in \{0, 1\}^n$ s.t. $x = a \Rightarrow \phi(a) = 1$?

k-SAT: each clause has width at most k .

Example: 4-SAT $\leq_p$ 3-SAT

Given a 4-CNF formula ϕ , construct a 3-CNF formula ψ such that ϕ is satisfiable $\iff \psi$ is satisfiable.

For each $c_j = (\ell_1 \vee \ell_2 \vee \ell_3 \vee \ell_4)$ in ϕ ,

$$c_{j,+} = (\ell_1 \vee \ell_2 \vee y_j)$$

$$c_{j,-} = (\ell_3 \vee \ell_4 \vee \neg y_j) \text{ for brand new } y_j$$

$$\psi = \bigwedge_{j=1}^m c_{j,+} \wedge c_{j,-} \quad \square$$

Loose ends

Still unclear

- ? why does the y_j trick preserve satisfiability in both directions?
- ? is the $k \rightarrow 3$ blowup really linear?

To do

- ✓ reread Sipser 7.4
- ✗ write up the k -SAT induction
- ! pset due Friday

Sipser, Thm 7.37

SAT is NP-complete. The proof simulates a nondeterministic polynomial-time machine with a tableau of configurations.

Bullets

- `\bul` – plain dot
- `\dash` – en dash
- ↳ `\arr` – the “sub-point” arrow
- ★ `\imp` – star, for things that matter
- ? `\qn` – something I did not follow
- ! `\bang` – warning to future me
- ✓ `\yes` and `\no`
- ✗ each takes an optional depth: `\bul[2]{...}`
- ↪ roll your own with `\newpoint{\hmm}{↪}`

Headings

`\head` – plain

`\headu` – ruled

`\headbox` – boxed

`\headbar` – filled bar

`\subhead` – small

Panels

Definition

A language L is **NP-hard** if every $L' \in \text{NP}$ reduces to it in polynomial time.

Key idea

Every panel takes an optional label: `\begin{idea}[My label]`. Pass `[]` to drop the label entirely.

Careful

The reduction has to run in polynomial time **in the size of the original input**, not the size of its

output.

Plain outlined box

outline and shade are the unlabelled generics.

Make your own: `\newshadebox{hmm}{mnviolet}{Hmm}` or `\newoutlinebox{...}`.

Side by side

Wider left

`\sbs[0.55]{left}{right}` splits two blocks, optionally unevenly.

one

$$\sum_i a_i$$

two

$$\bigcup_j B_j$$

Narrower right

- bullets work inside
- so do boxes and math

three

$$\vdash \bowtie$$

Emphasis

key term, alert, highlighted, muted aside, underlined, blue teal red amber violet gray

`\sep` draws that rule; `\gap` adds blank vertical space; `\eq{...}` sets left-aligned display math at the current indent.